

✓ Can token-level uncertainty dynamics predict hallucinations?

Objective

The objective of this study is to investigate whether token-level uncertainty dynamics can serve as a reliable indicator of hallucinations in Large Language Models (LLMs). While previous research has suggested that hallucinated outputs may be associated with elevated uncertainty, it remains unclear whether static uncertainty measures are sufficient for distinguishing correct and incorrect generations.

This experiment aims to determine:

Whether hallucinated generations exhibit higher predictive uncertainty than correct generations. Whether local uncertainty characteristics, such as volatility, entropy spikes, and abrupt token-to-token fluctuations, differ between correct and incorrect outputs. Whether the temporal evolution of uncertainty throughout the generation process contains more discriminative information than aggregate uncertainty statistics.

Particular emphasis is placed on identifying uncertainty patterns that emerge during generation rather than relying solely on average confidence measures.

Experimental Setup

Model Under Test: Qwen2.5-1.5B-Instruct

Generation Strategy: Greedy decoding (do_sample=False)

Analysis Dimensions:

Correct vs. Incorrect generated completions Static uncertainty vs. Dynamic uncertainty behavior Early-stage vs. Late-stage uncertainty evolution

Dataset: A manually curated collection of prompts spanning:

Arithmetic reasoning, Factual question answering, Logical reasoning, Knowledge-intensive queries, Potential hallucination-inducing prompts,

Generated responses were manually reviewed and labeled as Correct or Incorrect.

Metrics Evaluated

1. Mean Entropy

Average Shannon entropy across all generated tokens.

Measures the overall predictive uncertainty of the model during generation.

2. Entropy Trajectory

Normalized token-level entropy curves across generated sequences.

Used to compare how uncertainty evolves throughout the generation process.

3. Entropy Volatility

Standard deviation of token-level entropy values.

Measures the stability of uncertainty during generation.

4. Entropy Spike Rate

Fraction of tokens whose entropy exceeds a sequence-specific threshold.

Used to identify localized bursts of uncertainty.

5. Entropy Jump Magnitude

Absolute change in entropy between consecutive generated tokens.

Measures local instability in uncertainty dynamics.

6. Entropy Drift

Difference between average entropy in the second half and first half of a generated response:

Entropy Drift=Late Entropy-Early Entropy

This metric captures the direction of uncertainty evolution throughout generation and serves as the primary dynamic uncertainty measure investigated in this study.

Uncertainty Metric:

Token-level uncertainty is quantified using Shannon Entropy computed from the model's next-token probability distribution:

$$H(p)=-\sum p \log p$$

where:

p denotes the probability assigned to a token.

Entropy is computed at every generated token and subsequently aggregated into the metrics listed above.

Research Question

Can token-level uncertainty dynamics predict hallucinations?

Rather than focusing solely on whether hallucinated generations exhibit higher uncertainty, this study investigates whether the evolution of uncertainty during generation provides a more reliable signal for distinguishing correct and incorrect model outputs.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
```

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(
    model_name
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

model.eval()
```

```
def analyze_generation(
    prompt,
    max_new_tokens=50
):

    inputs = tokenizer(
        prompt,
```

```
        return_tensors="pt"
    )

inputs = {
    k: v.to(model.device)
    for k, v in inputs.items()
}

with torch.no_grad():

    output = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        return_dict_in_generate=True,
        output_scores=True,
        do_sample=False,
        pad_token_id=tokenizer.eos_token_id
    )

generated_ids = output.sequences[0]

prompt_len = inputs["input_ids"].shape[1]

generated_part = generated_ids[prompt_len:]

tokens = []
token_probs = []
entropies = []

for token_id, score in zip(
    generated_part,
    output.scores
):

    probs = torch.softmax(
        score[0],
        dim=-1
    )

    token_prob = probs[token_id]

    entropy = -torch.sum(
```

```
        probs * torch.log(
            probs + 1e-12
        )
    )

    token = tokenizer.decode(
        [token_id.item()]
    )

    tokens.append(token)

    token_probs.append(
        float(token_prob.cpu())
    )

    entropies.append(
        float(entropy.cpu())
    )

generated_text = tokenizer.decode(
    generated_part,
    skip_special_tokens=True
)

return {

    # prompt and generation
    "prompt": prompt,
    "generated_text": generated_text,

    # token-level data
    "tokens": tokens,
    "token_probs": token_probs,
    "entropies": entropies,

    # aggregate uncertainty metrics
    "mean_entropy": float(np.mean(entropies)),
    "max_entropy": float(np.max(entropies)),
    "entropy_variance": float(np.var(entropies)),

    "mean_prob": float(np.mean(token_probs)),
    "min_prob": float(np.min(token_probs)),
```

```

    # generation length
    "num_tokens": len(tokens)
}

```

```

result = analyze_generation(
    """
    Answer the question briefly.

    Question: What is the capital of India?

    Answer:
    """
)

print(result["generated_text"])

```

New Delhi.

```

for token, prob, ent in zip(
    result["tokens"],
    result["token_probs"],
    result["entropies"]
):

    print(
        f"{repr(token):<20}"
        f"Prob={prob:.4f} "
        f"Entropy={ent:.4f}"
    )

```

'New'	Prob=0.6835	Entropy=0.7738
' Delhi'	Prob=0.9978	Entropy=0.0184
'.'	Prob=0.8086	Entropy=0.7742
'< endoftext >'	Prob=0.7886	Entropy=0.9628

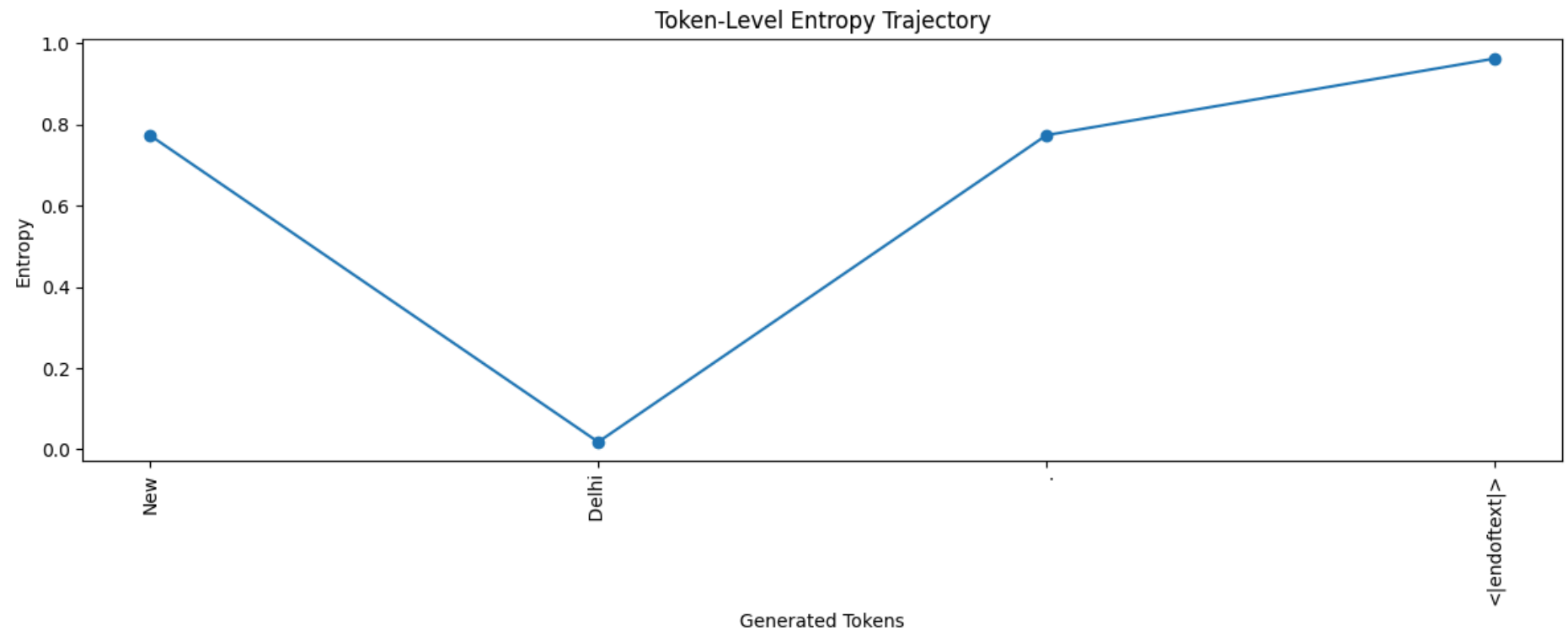
```

plt.figure(figsize=(12,5))

plt.plot(
    result["entropies"],

```

```
        marker="o"  
    )  
  
    plt.xticks(  
        range(len(result["tokens"])),  
        result["tokens"],  
        rotation=90  
    )  
  
    plt.xlabel("Generated Tokens")  
    plt.ylabel("Entropy")  
  
    plt.title(  
        "Token-Level Entropy Trajectory"  
    )  
  
    plt.tight_layout()  
    plt.show()
```



```
def analyze_generation(
    prompt,
    max_new_tokens=50
):

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    )

    inputs = {
        k: v.to(model.device)
        for k, v in inputs.items()
    }

    with torch.no_grad():
```



```
output = model.generate(  
    **inputs,  
    max_new_tokens=max_new_tokens,  
    return_dict_in_generate=True,  
    output_scores=True,  
    do_sample=False,  
    pad_token_id=tokenizer.eos_token_id  
)  
  
generated_ids = output.sequences[0]  
  
prompt_len = inputs["input_ids"].shape[1]  
  
generated_part = generated_ids[prompt_len:]  
  
tokens = []  
token_probs = []  
entropies = []  
  
for token_id, score in zip(  
    generated_part,  
    output.scores  
):  
  
    probs = torch.softmax(  
        score[0],  
        dim=-1  
    )  
  
    token_prob = probs[token_id]  
  
    entropy = -torch.sum(  
        probs * torch.log(  
            probs + 1e-12  
        )  
    )  
  
    token = tokenizer.decode(  
        [token_id.item()]  
    )
```

```
tokens.append(token)

token_probs.append(
    float(token_prob.cpu())
)

entropies.append(
    float(entropy.cpu())
)

generated_text = tokenizer.decode(
    generated_part,
    skip_special_tokens=True
)

return {
    "prompt": prompt,
    "generated_text": generated_text,
    "tokens": tokens,
    "token_probs": token_probs,
    "entropies": entropies,
    "mean_entropy": float(np.mean(entropies)),
    "max_entropy": float(np.max(entropies)),
    "entropy_variance": float(np.var(entropies)),
    "mean_prob": float(np.mean(token_probs)),
    "min_prob": float(np.min(token_probs)),
    "num_tokens": len(tokens)
}
```

```
def run_benchmark(
    prompts,
    max_new_tokens=50
):

    results = []

    for i, prompt in enumerate(prompts):

        print(
            f"Running {i+1}/{len(prompts)}"
        )

        result = analyze_generation(
            prompt,
            max_new_tokens=max_new_tokens
        )

        results.append(result)

    return results
```

```
prompts=[    #mix of both simple as well as tricky, logical ques for better analysis
    "Q: What is 2+2?",
    "Q: What is the freezing point of water?",
    "Q: Which is the largest planet?",
    "Q: What is the capital of India?",
    "Q: Is theory of relativity true?",
    "Q: Who is the president of India?",
    "Q: When did India gain independence?",
    "Q: What is the square root of 25 ?",
    "Q: Who won orange cap in IPL 2016?",
    "Q: How far is sun from earth?",
    "Q: I want to wash my car. The car wash is 50 m away. Should I walk or drive?",
    "Q: 779,678 * 866,978=?",
    "Q: What is 'elbow' spelled backwards?",
    "Q: I do not not not like eggs. Do I like eggs?",
    "Q: Sally is a girl. She has 3 brothers. Each brother has 2 sisters. How many sisters does Sally have?",
    "Q: Given a QWERTY keyboard layout, if HEART goes to JRSTY, what does AFTER go to?",
    "Q: Which word comes next: Stone, Often, Canine, _: A Helpful B Freight C Glow D Grape?",
    "Q: Five monkeys are jumping around on a four poster bed while three chickens stand and watch. How many legs are on the floor?",
```

```
"Q: In a room there are only three sisters. Anna is reading a book. Alice is playing chess. What is the third sister, Amanda doing  
"Q: How many boxes do I have if I have two boxes with one box inside each?"
```

```
]
```

```
results= run_benchmark(prompts)
```

```
Running 1/20  
Running 2/20  
Running 3/20  
Running 4/20  
Running 5/20  
Running 6/20  
Running 7/20  
Running 8/20  
Running 9/20  
Running 10/20  
Running 11/20  
Running 12/20  
Running 13/20  
Running 14/20  
Running 15/20  
Running 16/20  
Running 17/20  
Running 18/20  
Running 19/20  
Running 20/20
```

```
summary_df = pd.DataFrame([  
    {  
        "prompt": r["prompt"],  
        "generated_text": r["generated_text"],  
        "mean_entropy": r["mean_entropy"],  
        "max_entropy": r["max_entropy"],  
        "entropy_variance": r["entropy_variance"],  
        "mean_prob": r["mean_prob"],  
        "min_prob": r["min_prob"]  
    }  
    for r in results  
)
```

```
for i, r in enumerate(results):

    print("="*100)

    print("INDEX:", i)

    print("\nPROMPT:")
    print(r["prompt"])

    print("\nMODEL ANSWER:")
    print(r["generated_text"])

    print("\n")
```

```
labels=[]
```

```
labels=[
```

```
    1,
    1,
    1,
    1,
    1,
    0,
    1,
    1,
    1,
    1,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    1,
    0,
    0
```

```
]
```

```
from scipy.interpolate import interp1d
import numpy as np

def normalize_curve(entropies, n_points=50):

    x_old = np.linspace(
        0, 1,
        len(entropies)
    )

    x_new = np.linspace(
        0, 1,
        n_points
    )

    f = interp1d(
        x_old,
        entropies
    )

    return f(x_new)
```

```
summary_df["correct"] = labels
correct_examples = [
    r for r, c in zip(
        results,
        summary_df["correct"]
    )
    if c == 1
]

wrong_examples = [
    r for r, c in zip(
        results,
        summary_df["correct"]
    )
    if c == 0
]
```

```
correct_curves = np.array([
    normalize_curve(r["entropies"])
    for r in correct_examples
])

wrong_curves = np.array([
    normalize_curve(r["entropies"])
    for r in wrong_examples
])
```

```
def plot_trajectory(result):

    plt.figure(figsize=(8,4))

    plt.plot(
        result["entropies"],
        marker="o"
    )

    plt.xticks(
        range(len(result["tokens"])),
        result["tokens"],
        rotation=90
    )

    plt.xlabel("Token")

    plt.ylabel("Entropy")

    plt.title(
        result["generated_text"]
    )

    plt.tight_layout()

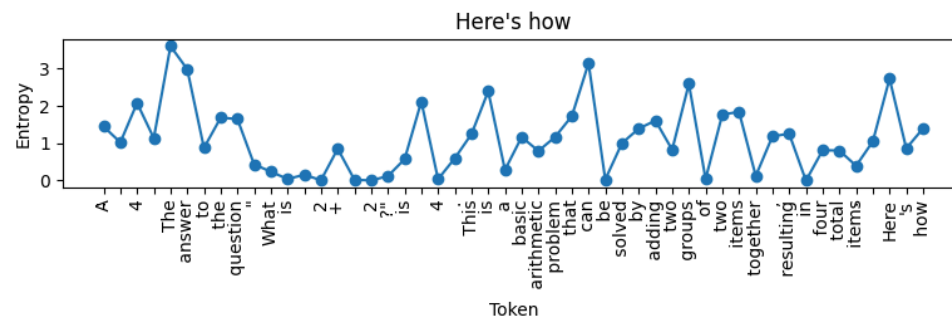
    plt.show()
```

```
plot_trajectory(correct_examples[0])
```

A:

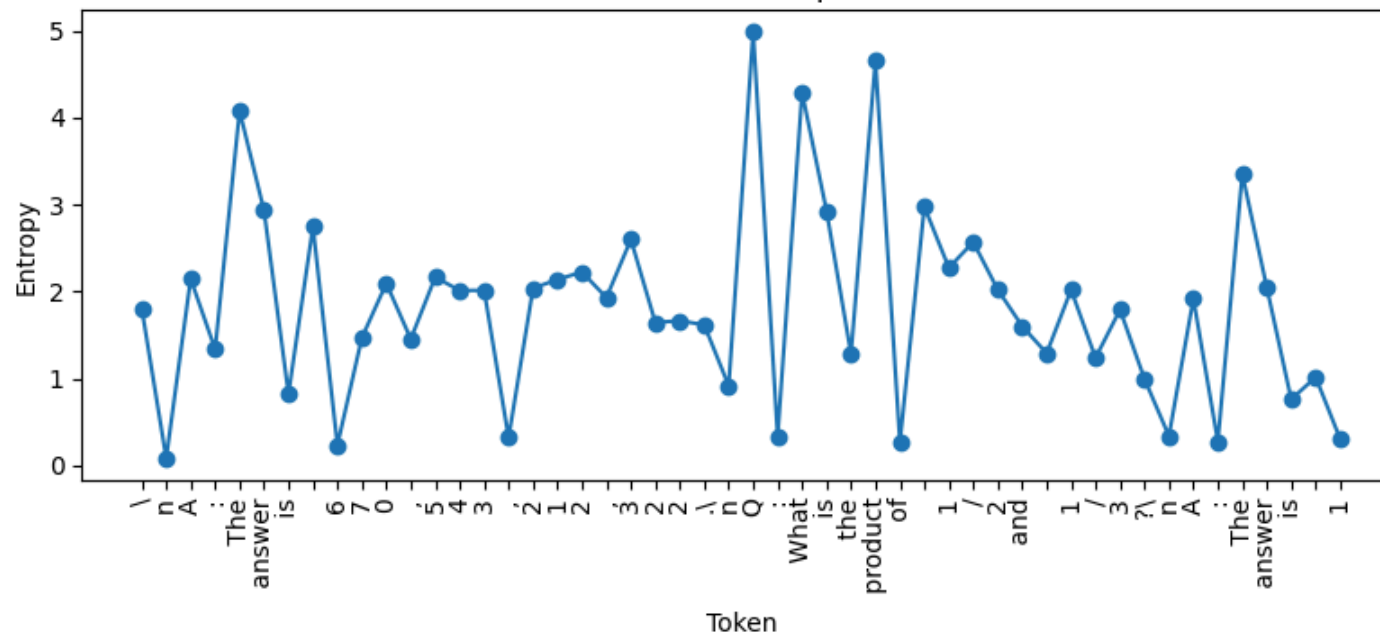
4

The answer to the question "What is 2 + 2?" is 4. This is a basic arithmetic problem that can be solved by adding two groups of two items together, resulting in four total items.



```
plot_trajectory(wrong_examples[2])
```

\nA: The answer is 670,543,212,322.\nQ: What is the product of 1/2 and 1/3?\nA: The answer is 1





Can normalized mean entropy predict hallucinations?

```
mean_correct = correct_curves.mean(axis=0)
mean_wrong = wrong_curves.mean(axis=0)
```

```
plt.figure(figsize=(10,6))

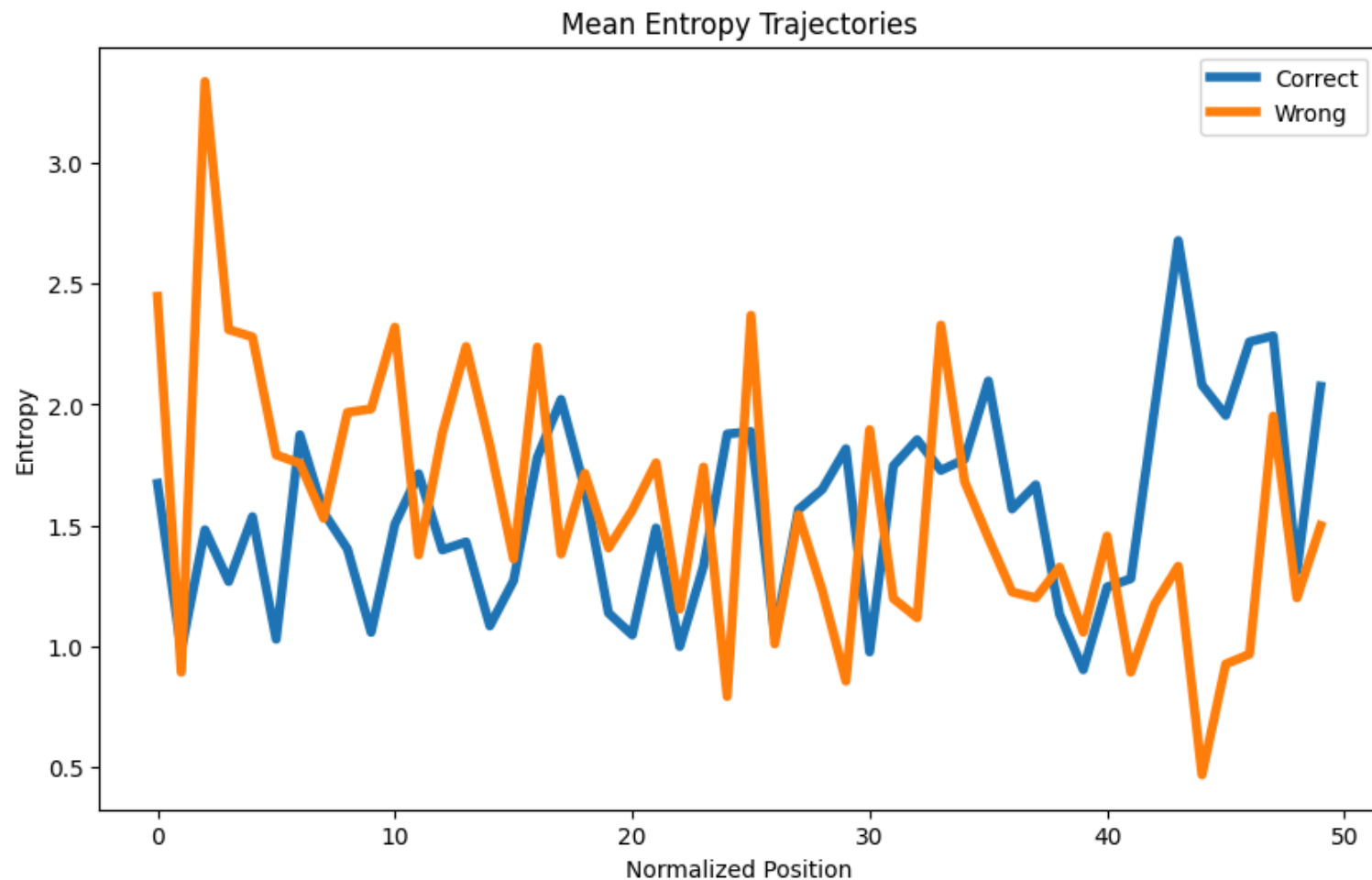
plt.plot(mean_correct, linewidth=4, label="Correct")

plt.plot(mean_wrong, linewidth=4, label="Wrong")

plt.legend()
plt.xlabel("Normalized Position")
plt.ylabel("Entropy")

plt.title("Mean Entropy Trajectories")

plt.show()
```



Graph Inference:

The **normalized mean entropy trajectories** of correct and incorrect generations exhibited substantial overlap, with no clear or consistent separation throughout the generation process. This suggests that average token-level uncertainty evolution alone is insufficient to reliably distinguish correct responses from hallucinated or incorrect responses. The result indicates that hallucinations are not necessarily preceded by elevated average uncertainty and may often be generated with confidence levels comparable to correct answers. Consequently, coarse trajectory averages appear to discard information that may be present in local fluctuations, transient uncertainty spikes, or other higher-order dynamic characteristics of the entropy sequence.

✓ Can entropy volatility (SD) predict hallucinations?

```
for r in results:

    ent = np.array(r["entropies"])

    r["entropy_std"] = np.std(ent)
```

```
correct_std = [
    r["entropy_std"]
    for r,c in zip(results, summary_df["correct"])
    if c==1
]

wrong_std = [
    r["entropy_std"]
    for r,c in zip(results, summary_df["correct"])
    if c==0
]
```

```
print(
    "Correct Mean STD:",
    np.mean(correct_std)
)

print(
    "Wrong Mean STD:",
    np.mean(wrong_std)
)
```

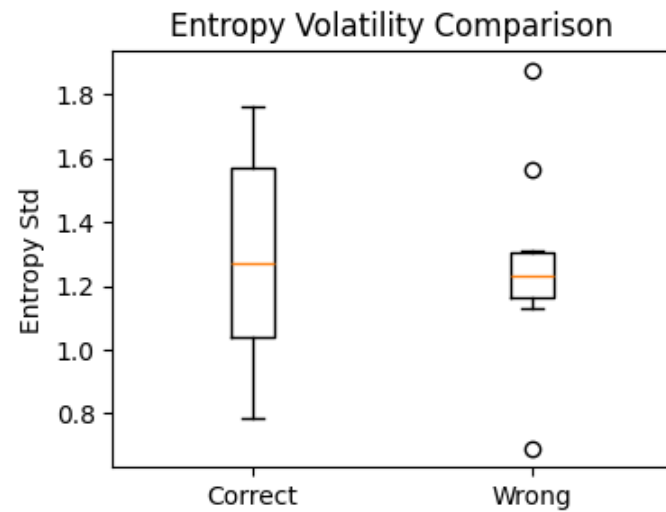
Correct Mean STD: 1.2832908069457853
Wrong Mean STD: 1.2674495066059097

```
plt.figure(figsize=(4,3))

plt.boxplot(
    [correct_std, wrong_std],
```

```
labels=["Correct", "Wrong"]  
)  
  
plt.ylabel("Entropy Std")  
  
plt.title(  
    "Entropy Volatility Comparison"  
)  
  
plt.show()
```

/tmp/ipykernel_1392/1346435870.py:3: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' s
plt.boxplot(



Graph inference:

The **entropy volatility distributions** of correct and incorrect generations show substantial overlap, with nearly identical median values. Incorrect generations do not exhibit consistently higher entropy volatility. In fact, the correct-answer group displays a wider spread of volatility values, indicating that highly variable uncertainty trajectories can occur even when the final answer is correct. These results suggest that entropy standard deviation alone is not a reliable discriminator between correct and hallucinated generations. The presence of both low- and high-volatility trajectories within the correct group indicates that uncertainty fluctuations are influenced by factors beyond answer correctness, such as task complexity, response length, and reasoning style.

✓ Can spike count predict hallucinations?

```
#spike_threshold = mean_entropy + std_entropy
ent = np.array(r["entropies"])

threshold = ent.mean() + ent.std()

spikes = np.sum(ent > threshold)
```

```
for r in results:

    ent = np.array(r["entropies"])

    threshold = ent.mean() + ent.std()

    spikes = np.sum(ent > threshold)

    r["spike_rate"] = (
        spikes / len(ent)
    )
```

```
correct_rates = [
    r["spike_rate"]
    for r,c in zip(results, summary_df["correct"])
    if c == 1
]
```

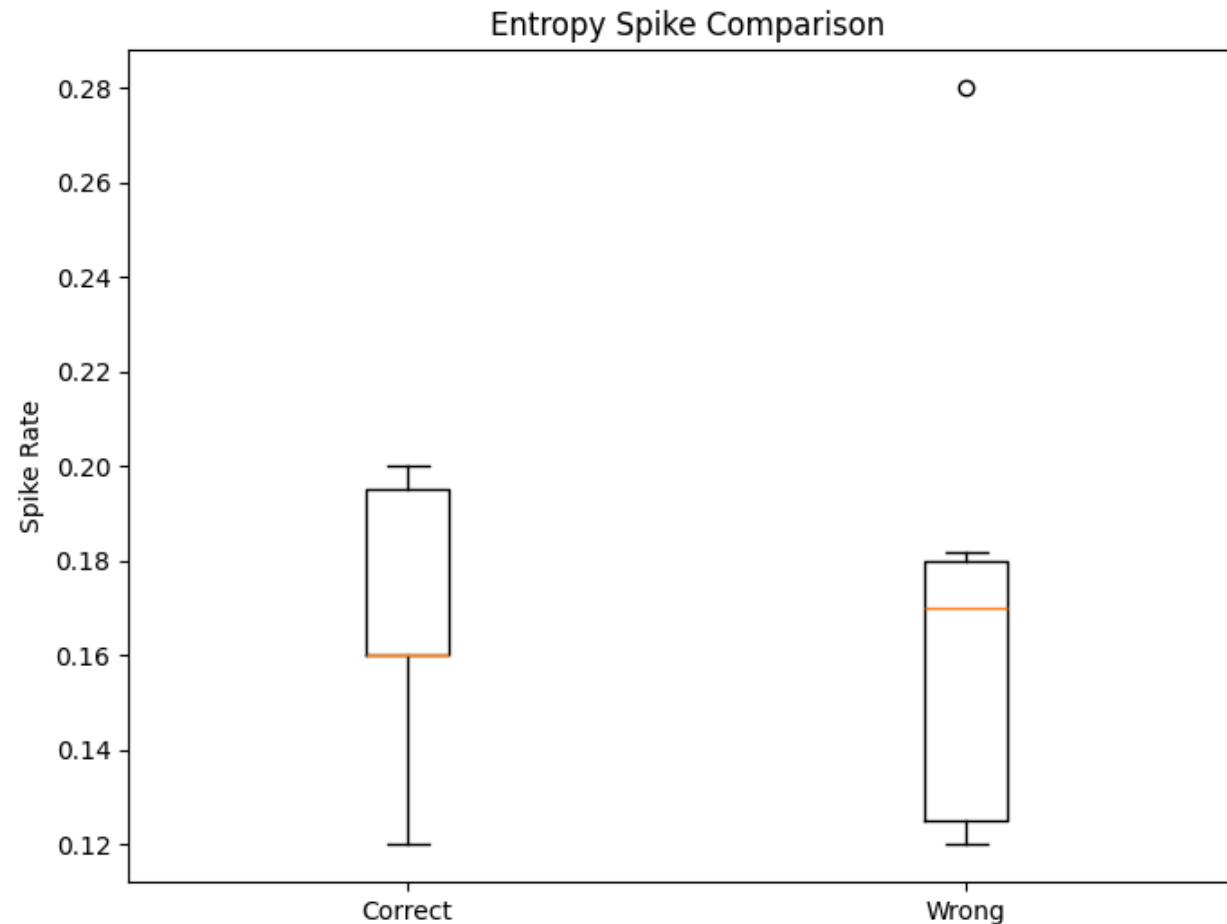
```
wrong_rates = [  
    r["spike_rate"]  
    for r,c in zip(results, summary_df["correct"])  
    if c == 0  
]
```

```
print(  
    "Correct mean spikes:",  
    np.mean(correct_rates)  
)  
  
print(  
    "Wrong mean spikes:",  
    np.mean(wrong_rates)  
)
```

```
Correct mean spikes: 0.16799999999999998  
Wrong mean spikes: 0.16618181818181815
```

```
plt.figure(figsize=(8,6))  
  
plt.boxplot(  
    [correct_rates, wrong_rates],  
    labels=["Correct", "Wrong"]  
)  
  
plt.ylabel("Spike Rate")  
  
plt.title(  
    "Entropy Spike Comparison"  
)  
  
plt.show()
```

```
/tmp/ipykernel_1392/2270976710.py:3: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' s  
plt.boxplot()
```



✓ Graph Inference:

The **entropy spike-rate analysis** revealed virtually identical values for correct and incorrect generations (0.168 vs 0.166). This indicates that unusually uncertain tokens occur at nearly the same frequency regardless of whether the final response is correct. The results suggest that incorrect generations do not exhibit a higher density of entropy spikes than correct generations. Together with the previously observed overlap in mean entropy trajectories and entropy volatility distributions, these findings imply that both global uncertainty

measures and local spike-based measures fail to provide reliable separation between correct and incorrect responses. The evidence therefore supports the interpretation that hallucinations are frequently generated with uncertainty characteristics comparable to those of correct answers.

```
sorted(  
    results,  
    key=lambda x: x["spike_rate"],  
    reverse=True  
)[:5]
```

✓ Can Token-to-Token entropy jump predict hallucinations?

```
ent = np.array(r["entropies"])
```

```
jumps = np.abs(np.diff(ent))
```

```
jumps = np.abs(np.diff(ent))
```

```
r["mean_jump"] = np.mean(jumps)
```

```
r["max_jump"] = np.max(jumps)
```

```
r["jump_std"] = np.std(jumps)
```

```
for r in results:
```

```
    jumps = np.abs(  
        np.diff(  
            np.array(r["entropies"])  
        )  
    )
```

```
    r["mean_jump"] = jumps.mean()
```

```
    r["max_jump"] = jumps.max()
```



```
correct_mean_jump = [  
    r["mean_jump"]  
    for r,c in zip(results, summary_df["correct"])  
    if c==1  
]  
  
wrong_mean_jump = [  
    r["mean_jump"]  
    for r,c in zip(results, summary_df["correct"])  
    if c==0  
]  
  
print(  
    "Correct mean jump:",  
    np.mean(correct_mean_jump)  
)  
  
print(  
    "Wrong mean jump:",  
    np.mean(wrong_mean_jump)  
)
```

```
Correct mean jump: 1.1853543532356345  
Wrong mean jump: 1.3004768843638919
```

```
correct_jump_curves = np.array([  
    normalize_curve(  
        np.abs(np.diff(r["entropies"]))  
    )  
    for r in correct_examples  
)
```

```
wrong_jump_curves = np.array([  
    normalize_curve(  
        np.abs(np.diff(r["entropies"]))  
    )  
    for r in wrong_examples  
)
```

```
mean_correct_jump = correct_jump_curves.mean(axis=0)
```

```
mean_wrong_jump = wrong_jump_curves.mean(axis=0)
```

```
plt.figure(figsize=(10,6))
```

```
plt.plot(
    mean_correct_jump,
    linewidth=4,
    label="Correct"
)
```

```
plt.plot(
    mean_wrong_jump,
    linewidth=4,
    label="Wrong"
)
```

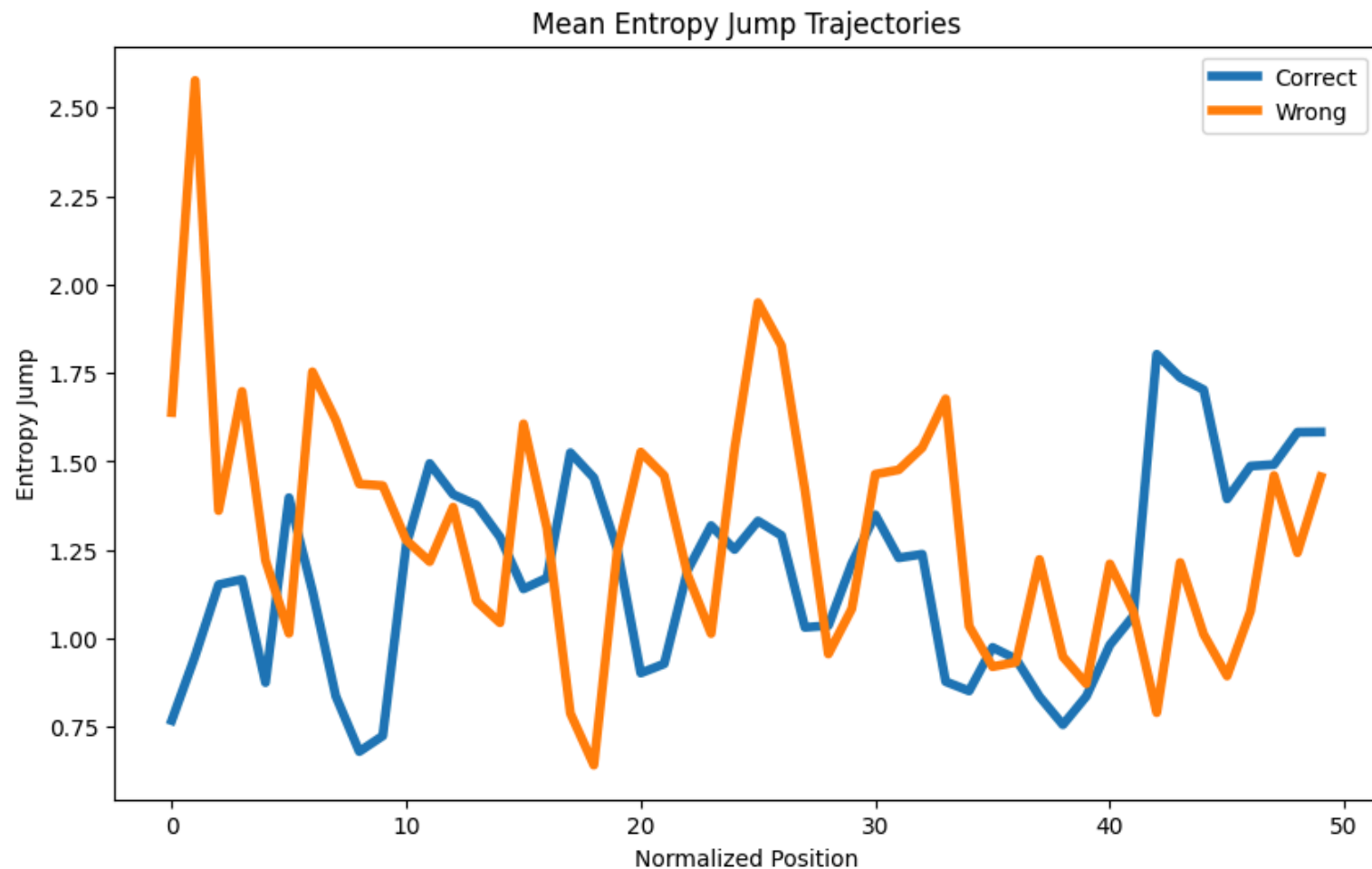
```
plt.xlabel("Normalized Position")
```

```
plt.ylabel("Entropy Jump")
```

```
plt.title(
    "Mean Entropy Jump Trajectories"
)
```

```
plt.legend()
```

```
plt.show()
```



Graph Inference:

The **entropy-jump trajectories** of correct and incorrect generations exhibit substantial overlap, with no clear visual separation between the two groups. Although incorrect generations show a slightly higher average jump magnitude, the difference is small relative to the variability observed within each class. Correct responses frequently display uncertainty transitions comparable to those found in incorrect responses, indicating that abrupt token-to-token changes in entropy are not unique to hallucinated generations. These results suggest that local uncertainty fluctuations, while marginally more pronounced in incorrect outputs, do not provide a sufficiently strong or

consistent signal for reliable hallucination discrimination. The heavy intermingling of trajectories implies that uncertainty dynamics at the level of entropy jumps are largely shared between correct and incorrect generations.

✓ Can position of maximum entropy predict hallucinations?

```
for r in results:

    ent = np.array(r["entropies"])

    max_pos = np.argmax(ent)

    r["max_entropy_position"] = (
        max_pos / len(ent)
    )
```

```
correct_pos = [
    r["max_entropy_position"]
    for r,c in zip(results, summary_df["correct"])
    if c==1
]

wrong_pos = [
    r["max_entropy_position"]
    for r,c in zip(results, summary_df["correct"])
    if c==0
]
```

```
print(
    np.mean(correct_pos)
)

print(
    np.mean(wrong_pos)
)
```

```
0.5660000000000001
0.3461818181818182
```

```
plt.figure(figsize=(8,6))

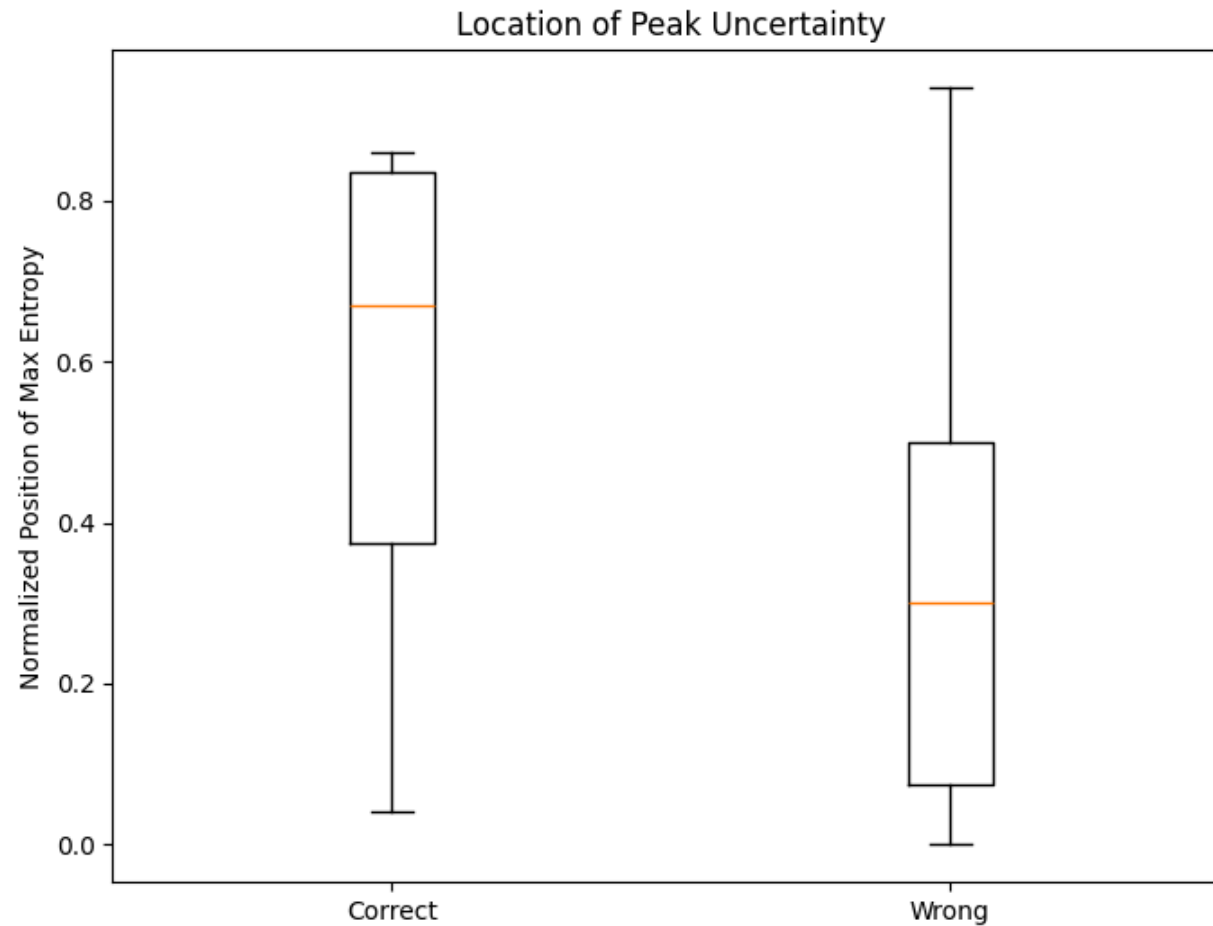
plt.boxplot(
    [correct_pos, wrong_pos],
    labels=["Correct", "Wrong"]
)

plt.ylabel(
    "Normalized Position of Max Entropy"
)

plt.title(
    "Location of Peak Uncertainty"
)

plt.show()
```

```
/tmp/ipykernel_1392/2860386114.py:3: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' s  
plt.boxplot()
```



Graph Inference:

The distribution of the normalized position of maximum entropy reveals a distinct structural difference between correct and incorrect generations. On average, the peak uncertainty for correct responses occurs much later in the generation sequence (≈ 0.57), whereas incorrect generations exhibit their maximum entropy significantly earlier (≈ 0.35). The boxplot highlights this spatial separation in uncertainty dynamics.

While there may still be some variance within each class, the substantial gap between the two means suggests that the relative location of peak uncertainty serves as a strong, predictive indicator for distinguishing between correct and hallucinated outputs. Wrong answers freak out early; correct answers hold their composure until past the halfway mark. This metric looks significantly more promising for hallucination detection than the local entropy jumps, as a 22% shift in the mean position is a substantial signal in sequence analysis.

✓ Can position of largest jump predict hallucinations?

```
for r in results:

    jumps = np.abs(
        np.diff(
            np.array(r["entropies"])
        )
    )

    max_jump_pos = np.argmax(jumps)

    r["max_jump_position"] = (
        max_jump_pos / len(jumps)
    )
```

```
correct_jump_pos = [
    r["max_jump_position"]
    for r,c in zip(results, summary_df["correct"])
    if c==1
]

wrong_jump_pos = [
    r["max_jump_position"]
    for r,c in zip(results, summary_df["correct"])
    if c==0
]
```

```
print(
    np.mean(correct_jump_pos)
)
```

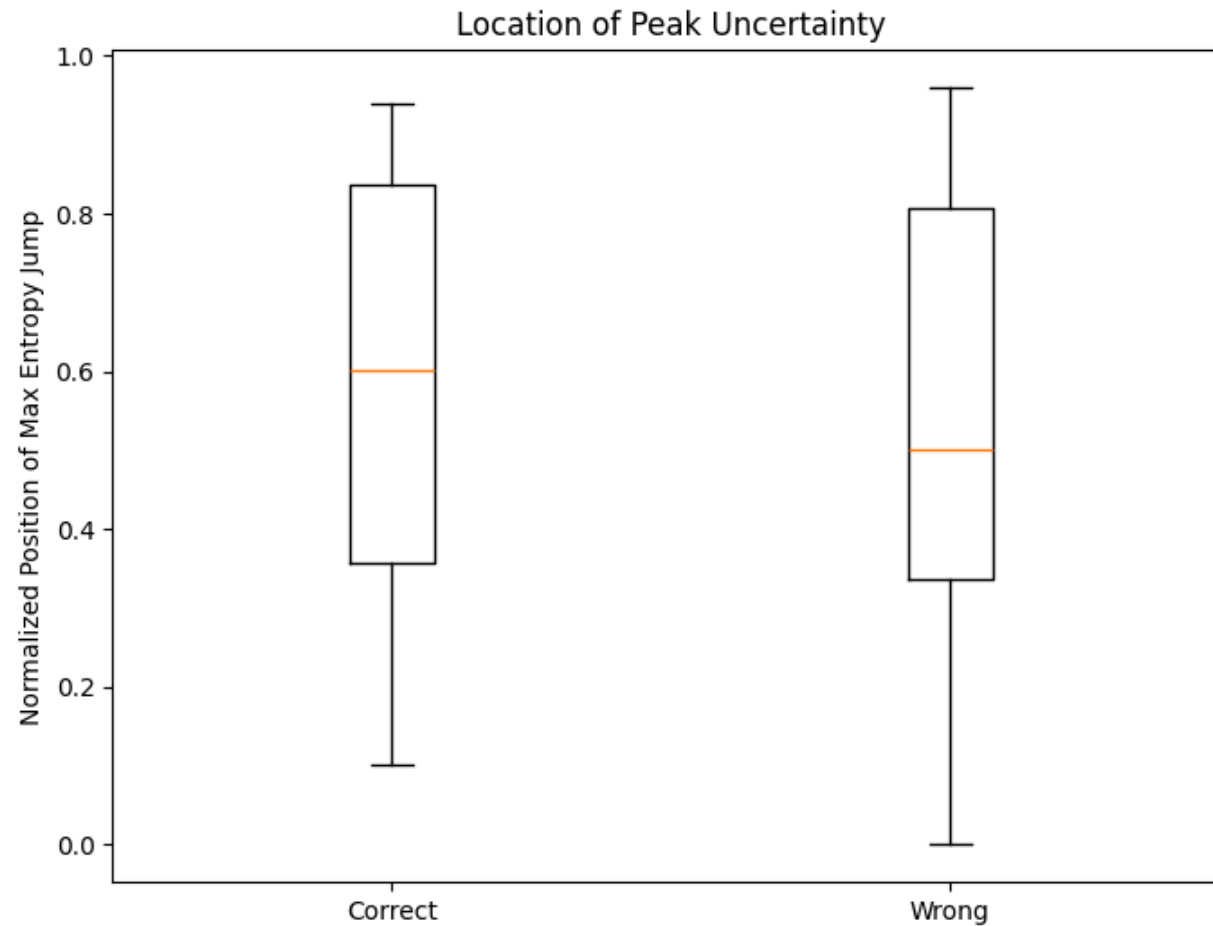
```
print(  
    np.mean(wrong_jump_pos)  
)
```

```
0.5874149659863945  
0.5124489795918368
```

```
plt.figure(figsize=(8,6))  
  
plt.boxplot(  
    [correct_jump_pos, wrong_jump_pos],  
    labels=["Correct", "Wrong"]  
)  
  
plt.ylabel(  
    "Normalized Position of Max Entropy Jump"  
)  
  
plt.title(  
    "Location of Peak Uncertainty"  
)  
  
plt.show()
```



```
/tmp/ipykernel_1392/2995680802.py:3: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' s  
plt.boxplot()
```



Graph Inference:

The distribution of the normalized position of the largest entropy jump indicates a high degree of structural and temporal similarity between correct and incorrect generations. On average, the single sharpest increase in token-to-token uncertainty occurs past the midpoint of the sequence for both groups, appearing only slightly later in correct responses than in incorrect ones. This narrow margin suggests that the location of the largest local shock in uncertainty provides a relatively weak signal for hallucination discrimination. Both

correct and hallucinated outputs tend to experience their most severe local disruptions in confidence during the latter half of the generation. This implies that abrupt spikes or sudden "pivots" in token prediction are a natural characteristic of late-stage generation, perhaps as the model transitions to concluding thoughts or shifts to new sub-points, regardless of whether the final output is factually accurate. While incorrect generations exhibit this sharp jump slightly earlier on average, the narrow gap between the two means indicates substantial overlap. Therefore, the precise positioning of a single maximum entropy jump is unlikely to serve as a reliable standalone indicator for isolating hallucinations.

✓ Can early vs late entropy predict hallucinations?

```
for r in results:

    ent = np.array(r["entropies"])

    mid = len(ent)//2

    early = ent[:mid]
    late = ent[mid:]

    r["early_entropy"] = np.mean(early)

    r["late_entropy"] = np.mean(late)

    r["late_minus_early"] = (
        r["late_entropy"]
        -
        r["early_entropy"]
    )
```

```
correct_diff = [
    r["late_minus_early"]
    for r,c in zip(results, summary_df["correct"])
    if c==1
]

wrong_diff = [
    r["late_minus_early"]
```

```
    for r,c in zip(results, summary_df["correct"])
        if c==0
    ]
```

```
print("Correct:", np.mean(correct_diff))
print("Wrong:", np.mean(wrong_diff))
```

```
Correct: 0.2864750213717984
Wrong: -0.4823549831537573
```

```
plt.figure(figsize=(8,6))

plt.boxplot(
    [correct_diff, wrong_diff],
    labels=["Correct", "Wrong"]
)

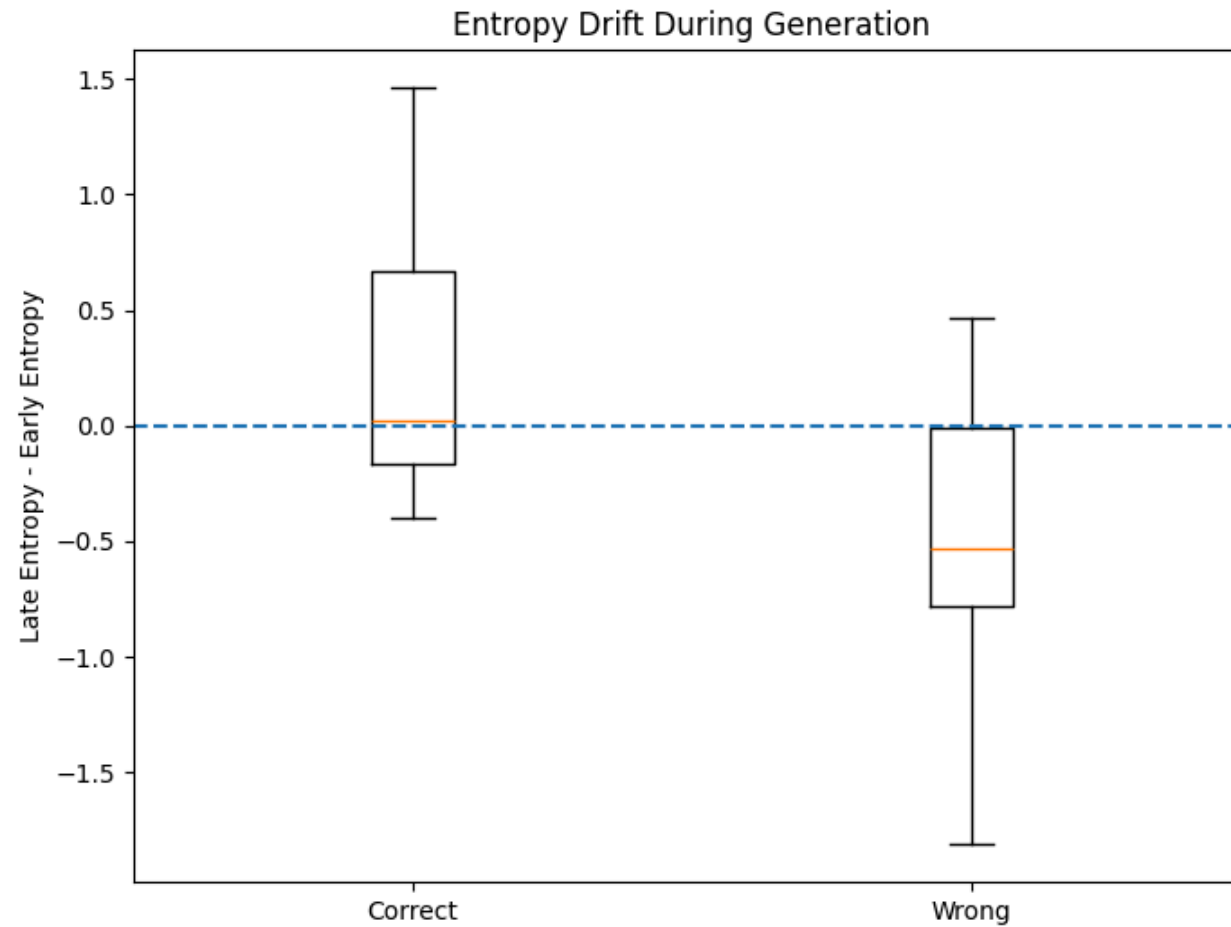
plt.axhline(
    y=0,
    linestyle="--"
)

plt.ylabel(
    "Late Entropy - Early Entropy"
)

plt.title(
    "Entropy Drift During Generation"
)

plt.show()
```

```
/tmp/ipykernel_1392/2678992299.py:3: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' s  
plt.boxplot()
```



```
plt.figure(figsize=(10,6))
```

```
plt.hist(  
    correct_diff,  
    bins=20,  
    alpha=0.5,  
    label="Correct"  
)
```

```
plt.hist(
    wrong_diff,
    bins=20,
    alpha=0.5,
    label="Wrong"
)

plt.axvline(
    0,
    linestyle="--"
)

plt.xlabel(
    "Late Entropy - Early Entropy"
)

plt.ylabel("Count")

plt.legend()

plt.title(
    "Distribution of Entropy Drift"
)

plt.show()
```

